ME-2 Digital Twin Baseline Construction

Week 9: Twin Output Definition and AI Interface Specification

Blayne Montaque

Spring 2026

1 Purpose

The purpose of this stage is to define the outputs of the behavioral digital twin in a form that can be consumed directly by the CpE telemetry, cybersecurity, and AI-monitoring pipeline.

Earlier stages of the project established:

- Static baseline modeling
- Dynamic vibration characterization
- Combined static and dynamic response analysis
- MATLAB-based signal processing workflows
- Experimental sensor synchronization and acquisition

This stage converts those earlier modeling efforts into a structured interface layer that can support:

- AI-based anomaly detection
- Timing-aware monitoring
- Cyber-physical behavioral validation
- Residual-based condition monitoring
- Intelligent telemetry analysis

The overall objective is to define what the digital twin should output, how those outputs are computed, and how the outputs can be integrated into the larger CpE monitoring architecture.

2 Relationship to CpE System Architecture

The behavioral digital twin is intended to operate downstream of the sensing, telemetry, and AI infrastructure being developed by the CpE teams.

The CpE workflow focuses primarily on secure sensing, trusted communication, telemetry integrity, timing awareness, and AI-based monitoring. The ME digital twin complements that work by providing a physics-informed behavioral reference for the structural system.

Rather than only transmitting raw sensor values, the digital twin introduces expected-response behavior into the pipeline. This allows the larger cyber-physical system to evaluate not only whether data packets are valid and authenticated, but also whether the measured structural behavior itself is physically reasonable.

The relationship between the major subsystems can be summarized as:

Sensors → Embedded Acquisition → Secure Telemetry → Digital Twin → AI Monitoring Layer

Within this architecture, the digital twin receives synchronized sensor measurements and computes:

- Expected structural response
- Statistical operating bounds
- Residual error signals
- Dynamic-response metrics
- Frequency-domain features

These outputs can then be consumed by the CpE AI pipeline for anomaly detection, timing analysis, replay awareness, and behavioral verification.

3 Supporting Modeling Work

The twin-output specification developed in this stage is based on the MATLAB modeling and experimental implementations developed during Weeks 5 through 8.

The supporting files include:

Table 1: Supporting Modeling Files

File	Purpose
`static_ss_model.m`	Static baseline modeling implementation used to generate expected-value relationships, residuals, and tolerance bounds.
`sensor_test.m`	Dynamic-response processing implementation used for FFT analysis, acceleration processing, and vibration feature extraction.
`data.csv`	Experimental dataset containing synchronized structural-response measurements.
`load_cell_test.ino`	Arduino-based embedded acquisition implementation used for synchronized sensing and vibration excitation.

These implementations collectively form the computational and experimental foundation of the digital twin.

4 Digital Twin Output Philosophy

The behavioral digital twin is not intended to function as a high-fidelity finite element solver.

Instead, the twin acts as a behavioral reference model for the structure.

The twin attempts to answer the following question:

Given the current operating conditions, what response should the structure produce?

The digital twin therefore compares:

$$y_{\text{measured}}$$

against:

$$y_{\text{expected}}$$

and computes the difference between them.

The resulting deviation metrics become inputs to the AI-monitoring layer.

5 Primary Twin Outputs

The current digital twin produces four primary output categories:

1. Expected-value outputs
2. Statistical operating bounds
3. Residual signals
4. Derived dynamic-response features

These outputs are intended to be transmitted alongside the telemetry stream so that downstream AI systems can evaluate structural behavior continuously.

6 Expected-Value Outputs

The first digital twin output is the expected structural response.

The expected-value relationship is generated using the MATLAB baseline models developed during the earlier static and dynamic studies.

The general relationship can be written as:

$$y_{\text{expected}} = f(L, E, T)$$

where:

- L represents applied loading
- E represents vibration excitation magnitude
- T represents environmental conditioning variables

The static baseline MATLAB implementation currently approximates this behavior using:

$$\hat{y} = ax + b$$

This relationship was implemented directly in:

`static_ss_model.m`

using:

`polyfit(x,y,1)`

The expected-value output allows the AI pipeline to compare incoming measurements against the baseline structural behavior predicted by the digital twin.

7 Statistical Operating Bounds

The second digital twin output category is the acceptable operating range around the expected response.

The MATLAB implementation defines the tolerance envelope using:

$$\text{tol} = k\sigma$$

where:

- k is the tolerance multiplier
- σ is the baseline-response standard deviation

The current implementation uses:

$$k = 2$$

The upper and lower operating limits are therefore:

$$y_{\text{upper}} = y_{\text{expected}} + \text{tol}$$

$$y_{\text{lower}} = y_{\text{expected}} - \text{tol}$$

These bounds define the region of acceptable structural behavior under baseline operating conditions.

The AI-monitoring layer can therefore determine whether measured behavior remains inside or outside the expected operating envelope.

8 Residual Output

The residual signal is currently the most important output generated by the digital twin.

The residual is defined as:

$$r(t) = y_{\text{measured}}(t) - y_{\text{expected}}(t)$$

This relationship is already implemented throughout the MATLAB modeling workflow.

For example, in:

`static_ss_model.m`

the residual is computed directly using:

```
res = y - yFit;
```

The residual compresses the structural behavior into a single deviation-oriented signal. Rather than processing raw measurements alone, the AI system can analyze:

- Residual magnitude
- Residual drift
- Residual persistence
- Residual oscillation
- Residual growth rate

These features are significantly more useful than raw sensor values because they already contain baseline behavioral context.

9 Dynamic-Response Outputs

The dynamic MATLAB implementation also produces vibration-related outputs for AI analysis.

The resultant acceleration magnitude is computed using:

$$a_{\text{mag}} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

This operation is implemented directly in:

`sensor_test.m`

using:

```
AccelMag = sqrt(Xacc.^2 + Yacc.^2 + Zacc.^2);
```

The FFT implementation then produces frequency-domain features using:

$$Y = \texttt{fft}(a_{\text{detrended}})$$

The dynamic-response outputs include:

- RMS acceleration
- Dominant frequency

- FFT peak magnitude
- Frequency shifts
- Oscillation persistence
- Broadband vibration behavior

These outputs are important because cyber-physical disturbances may appear as abnormal timing behavior, unexpected oscillation, or unusual frequency-domain response.

10 AI-Oriented Deviation Metrics

The CpE AI systems require structured numerical features rather than raw sensor streams alone.

The digital twin therefore exports deviation-oriented behavioral metrics.

Table 2: AI-Oriented Twin Outputs

Metric	**Equation**	**Purpose**
Residual Magnitude	$\lvert r(t) \rvert$	Measures instantaneous behavioral deviation
Normalized Residual	$\frac{r(t)}{\sigma}$	Measures deviation relative to baseline variability
Residual RMS	r_{RMS}	Measures long-duration deviation intensity
Frequency Shift	$\Delta f = f_{\text{measured}} - f_{\text{baseline}}$	Measures changes in vibration behavior
Bound Violation Flag	$y \notin [y_L, y_U]$	Indicates operation outside expected bounds
Timing Deviation	$\Delta t_{\text{arrival}}$	Supports timing-aware monitoring

These outputs are intentionally structured so they can be consumed directly by the CpE telemetry, timing-monitoring, and AI-analysis pipeline.

11 Cybersecurity Relevance

The digital twin provides behavioral verification for the cyber-physical monitoring system.

A telemetry packet may be cryptographically valid while still representing physically unrealistic behavior.

For example:

- Replay attacks may resend old but valid data
- Delayed telemetry may alter timing behavior

- Spoofed sensors may transmit unrealistic values
- Corrupted firmware may produce inconsistent measurements

The digital twin therefore introduces a second validation layer:

$$\text{Cyber-valid} \neq \text{Physically-valid}$$

The AI-monitoring system can use residuals and behavioral deviation metrics to determine whether the measured structural behavior remains physically reasonable.

This creates a direct connection between cybersecurity and structural-response validation.

12 Interface Notes for AI Pipeline Integration

The digital twin outputs should be transmitted using a structured format compatible with the telemetry and AI-monitoring pipeline.

A recommended output structure is shown below.

Table 3: Recommended Twin Output Schema

Field	**Type**	**Purpose**
Timestamp	Float	Synchronization with telemetry pipeline
Expected Load	Float	Twin-predicted structural response
Measured Load	Float	Measured sensor response
Residual	Float	Difference between expected and measured behavior
Residual RMS	Float	Long-window deviation metric
Dominant Frequency	Float	Primary vibration-response feature
FFT Peak Magnitude	Float	Frequency-domain response intensity
Bound Violation Flag	Boolean	Indicates behavior outside operating bounds
Timing Anomaly Flag	Boolean	Supports timing-aware monitoring

This output structure allows the AI pipeline to evaluate both cyber integrity and structural-behavior integrity simultaneously.

13 Future Development Direction

Future development stages can expand the digital twin further by incorporating:

- Temperature-conditioned baseline models
- Adaptive thresholds
- Online recursive updating
- Sensor trust weighting
- Machine-learning-generated expected values
- Multi-state operating classification

These additions would allow the digital twin to evolve into a continuously adaptive cyber-physical behavioral model.

14 Summary

Week 9 defines the operational outputs of the behavioral digital twin and establishes how those outputs integrate with the larger telemetry, cybersecurity, and AI-monitoring architecture.

The MATLAB modeling work completed throughout earlier stages now provides:

- Expected-response prediction
- Statistical operating bounds
- Residual generation
- Frequency-domain characterization
- AI-consumable behavioral metrics

These outputs allow the digital twin to function as a behavioral verification layer for the larger cyber-physical monitoring system.

A Appendix A: Associated Project Files

The following files support the twin-output workflow described throughout this report.

Table 4: Associated Digital Twin Files

File	Purpose
`static_ss_model.m`	Static baseline modeling implementation used for expected-value generation and residual computation.
`sensor_test.m`	Dynamic-response processing implementation used for FFT analysis and vibration feature extraction.
`data.csv`	Experimental synchronized sensor dataset used for baseline-response characterization.
`load_cell_test.ino`	Arduino-based acquisition implementation used for synchronized sensing and vibration excitation.

All associated implementations, datasets, and deliverables have been uploaded to the shared GitHub repository for reproducibility and continued AI-pipeline integration.